

An Analysis of Q-learning

Q-learning

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

Q-learning -- Part 1

Discount factor The discount factor γ determines the importance of future rewards. A factor of 0 will make the agent "myopic" (or short-sighted) by only considering current rewards, i.e. r_t (in the update rule above), while a factor approaching 1 will make it strive for a long-term high reward. If the discount factor meets or exceeds 1, the action values may diverge. For $\gamma = 1$, without a terminal state, or if the agent never reaches one, all environment histories become infinitely long, and utilities with additive, undiscounted rewards generally become infinite. Even with a discount factor only slightly lower than 1, Q-function learning leads to propagation of errors and instabilities when the value function is approximated with an artificial neural network. In that case, starting with a lower discount factor and increasing it towards its final value accelerates learning.

Q-learning -- Part 2

5 second wait time + 0 second fight time Through exploration, despite the initial (patient) action resulting in a larger cost (or negative reward) than in the forceful strategy, the overall cost is lower, thus revealing a more rewarding strategy.

Q-learning -- Part 3

Multi-agent learning Q-learning has been proposed in the multi-agent setting (see Section 4.1.2 in). One approach consists in pretending the environment is passive. Littman proposes the minimax Q learning algorithm.

Q-learning -- Part 4

where R_{t+1} is the reward received when moving from the state S_t to the state S_{t+1} , and α is the learning rate ($0 < \alpha \leq 1$). Note that $Q_{new}(S_t, A_t)$ is the sum of three terms:

Q-learning -- Part 5

Q-learning is a reinforcement learning algorithm that trains an agent to assign values to its possible actions based on its current state, without requiring a model of the environment (model-free).

It can handle problems with stochastic transitions and rewards without requiring adaptations. For example, in a grid maze, an agent learns to reach an exit worth 10 points. At a junction, Q-learning might assign a higher value to moving right than left if right gets to the exit faster, improving this choice by trying both directions over time. For any finite Markov decision process, Q-learning finds an optimal policy in the sense of maximizing the expected value of the total reward over any and all successive steps, starting from the current state. Q-learning can identify an optimal action-selection policy for any given finite Markov decision process, given infinite exploration time and a partly random policy. "Q" refers to the function that the algorithm computes: the expected reward—that is, the quality—of an action taken in a given state.

Q-learning -- Part 6

Function approximation Q-learning can be combined with function approximation. This makes it possible to apply the algorithm to larger problems, even when the state space is continuous. One solution is to use an (adapted) artificial neural network as a function approximator. Another possibility is to integrate Fuzzy Rule Interpolation (FRI) and use sparse fuzzy rule-bases instead of discrete Q-tables or ANNs, which has the advantage of being a human-readable knowledge representation form. Function approximation may speed up learning in finite problems, due to the fact that the algorithm can generalize earlier experiences to previously unseen states.

Q-learning -- Part 7

0 seconds wait time + 15 seconds fight time On the next day, by random chance (exploration), you decide to wait and let other people depart first. This initially results in a longer wait time. However, less time is spent fighting the departing passengers. Overall, this path has a higher reward than that of the previous day, since the total boarding time is now:

Q-learning -- Part 8

Deep Q-learning The DeepMind system used a deep convolutional neural network, with layers of tiled convolutional filters to mimic the effects of receptive fields. Reinforcement learning is unstable or divergent when a nonlinear function approximator such as a neural network is used to represent Q. This instability comes from the correlations present in the sequence of observations, the fact that small updates to Q may significantly change the policy of the agent and the data distribution, and the correlations between Q and the target values. The method can be used for stochastic search in various domains and applications. The technique used experience replay, a

biologically inspired mechanism that uses a random sample of prior actions instead of the most recent action to proceed. This removes correlations in the observation sequence and smooths changes in the data distribution. Iterative updates adjust Q towards target values that are only periodically updated, further reducing correlations with the target.

Q-learning -- Part 9

Implementation Q-learning at its simplest stores data in tables. This approach falters with increasing numbers of states/actions since the likelihood of the agent visiting a particular state and performing a particular action is increasingly small.

Q-learning -- Part 10

Now the estimated value of the discounted future is evaluated using a different policy, which solves the overestimation issue. This algorithm was later modified in 2015 and combined with deep learning, as in the DQN algorithm, resulting in Double DQN, which outperforms the original DQN algorithm.

Q-learning -- Part 11

After Δt steps into the future the agent will decide some next step. The weight for this step is calculated as $\gamma^{\Delta t}$, where γ (the discount factor) is a number between 0 and 1 ($0 \leq \gamma \leq 1$). Assuming $\gamma < 1$, it has the effect of valuing rewards received earlier higher than those received later (reflecting the value of a "good start"). γ may also be interpreted as the probability to succeed (or survive) at every step Δt . The algorithm, therefore, has a function that calculates the quality of a state-action combination:

Q-learning -- Part 12

$$Q_{t+1}(s_t, a_t) = Q_t(s_t, a_t) + \alpha_t(s_t, a_t) (r_t + \gamma \max_a Q_t(s_{t+1}, a) - Q_t(s_t, a_t))$$

$$Q_{t+1}^A(s_t, a_t) = Q_t^A(s_t, a_t) + \alpha_t(s_t, a_t) \left(r_t + \gamma \max_a Q_t^B(s_{t+1}, a) - Q_t^A(s_t, a_t) \right)$$

, and